Первое приложение на Jetpack Compose



Activity, интерфейс и состояние

Разработка мобильных приложений на Android

Что должно стать понятным

- Как Android запускает наш код и зачем нужна Activity
- Как composable-функции описывают интерфейс
- Как состояние связано с тем, что видно на экране
- Как собрать один интерактивный экран

Результат занятия: экран персонажа с именем, здоровьем и действиями.

Инструменты: кто за что отвечает

Инструмент	Его задача
Android Studio	Редактирование, подсказки, Preview и отладка
Android SDK	API платформы, инструменты и образы устройств
Gradle и Android-плагин	Сборка проекта и подключение библиотек
Эмулятор или телефон	Установка и запуск приложения

Android Studio запускает инструменты, которые выполняют сборку и запуск.

Ориентируемся в проекте

Где	Что ищем
app	Модуль нашего приложения
MainActivity.kt	Activity и установка корневого интерфейса
Другие .kt-файлы	Компоненты экрана и обычная Kotlin-логика
ui/theme	Тема приложения, цвета и типографика
build.gradle.kts	Настройки сборки и зависимости модуля

Для первого экрана работаем в app: Activity, тема и собственные .kt-файлы.

Зависимости и настройки Android

- Compose и Material 3 подключаются как библиотеки
- Gradle Sync читает конфигурацию и получает зависимости
- minSdk задаёт минимальную поддерживаемую версию Android
- compileSdk задаёт доступные при сборке API платформы
- targetSdk влияет на применяемые правила совместимости

Версия Android, версия Kotlin и версия Compose — разные настройки.

Первый запуск проекта

- Создать проект из шаблона Empty Activity с Compose
- Дождаться завершения Gradle Sync
- Выбрать эмулятор или подключённый телефон
- Запустить app и убедиться, что базовый экран открывается

Сначала запускаем неизменённый шаблон. Потом добавляем свой экран.

Нажали на иконку. Что дальше?

Мы написали Kotlin-функции.
Кто даст им окно на телефоне?

Кто сообщит, что приложение открыли, свернули или вернулись в него?

Знакомая ситуация: страница в браузере

Среда показа

Браузер открывает страницу во вкладке.

У вкладки есть место на экране и своя жизнь.

Содержимое страницы

Внутри находятся заголовки, поля ввода и кнопки.

Их внешний вид и поведение описывает код страницы.

Разделяем среду, которая показывает интерфейс, и само содержимое.

Переносим аналогию на Android

Знакомый образ	Что полезно понять в Android
Браузер управляет показом страницы	Android управляет жизнью Activity
У страницы есть место для интерфейса	Activity предоставляет окно для UI
Внутри страницы находятся элементы	Compose описывает содержимое окна
Содержимое может меняться на месте	Разные экраны могут жить в одной Activity

Аналогия помогает разделить роли. Механизмы браузера и Android различаются.

Activity: окно под управлением Android

- Activity — компонент приложения для взаимодействия с пользователем
- Она предоставляет окно, в котором приложение показывает UI
- Android создаёт Activity и сообщает ей об этапах её жизни
- В нашем приложении содержимое окна описывает Compose

Рабочая модель: Android управляет Activity, Compose описывает её содержимое.

MainActivity: Android вызывает наш код

```
class MainActivity : ComponentActivity() {
    override fun onCreate(state: Bundle?) {
        super.onCreate(state)

        setContent {
            MyApp()
        }
    }
}
```

ComponentActivity — готовая базовая Activity.

onCreate вызывается при её создании.

setContent задаёт UI.

MyApp — наша composable-функция, которую объявим дальше.

Как читать MainActivity

Строка	Смысл
: ComponentActivity()	Наследуем готовое поведение Activity
override fun onCreate(...)	Переопределяем метод, который вызовет Android
super.onCreate(state)	Даём базовому классу выполнить его настройку
setContent { ... }	Устанавливаем корневое Compose-содержимое
MyApp()	Вызываем собственный UI-компонент

MainActivity не нужно создавать вручную через MainActivity().

Одна Activity, несколько экранов

MainActivity

Одно окно приложения.

В нём установлено Compose-содержимое.

Содержимое окна

Каталог

Карточка товара

Корзина

Новый экран приложения не требует автоматически новой Activity.

Пользователь нажал Home

Приложение исчезло с переднего плана. Оно обязательно закрылось?

А при возвращении обязательно снова вызовется onCreate()?

Жизнью Activity управляет система

Ситуация	Что важно сейчас
Первое создание экземпляра	Android вызывает onCreate
Окно стало невидимым	Activity может остаться в памяти
Возвращение к прежнему экземпляру	onCreate повторно не требуется
Пересоздание Activity	Создаётся новый экземпляр и вызывается onCreate
Завершение фонового процесса	Данные только в памяти могут потеряться

Рекомпозиция Compose и пересоздание Activity — разные события.

Jetpack Compose

- Набор библиотек для создания интерфейса на Kotlin
- Интерфейс собирается из composable-функций
- Функции получают данные и описывают элементы UI
- Compose отслеживает наблюдаемое состояние и обновляет UI

Кнопка, строка текста и целый экран могут быть composable-функциями.

Как Compose получает интерфейс



Описать элементы, определить их размеры и положение, нарисовать.

Интерфейс зависит от данных

```
@Composable
fun HealthText(hp: Int) {
    if (hp > 0) {
        Text("Здоровье: $hp")
    } else {
        Text("Персонаж побеждён")
    }
}
```

hp = 100:
«Здоровье: 100»

hp = 0:
«Персонаж побеждён»

Мы описываем, какой UI нужен для текущего hp.

Первая composable-функция

```
@Composable
fun Greeting(name: String) {
    Text(text = "Привет, $name!")
}

// Внутри setContent или другой composable:
// Greeting(name = "Аня")
```

@Composable помечает функцию для Compose.

Text показывает текст на экране.

UI-функции с результатом Unit обычно называют с большой буквы.

Фигурные скобки здесь уже знакомы

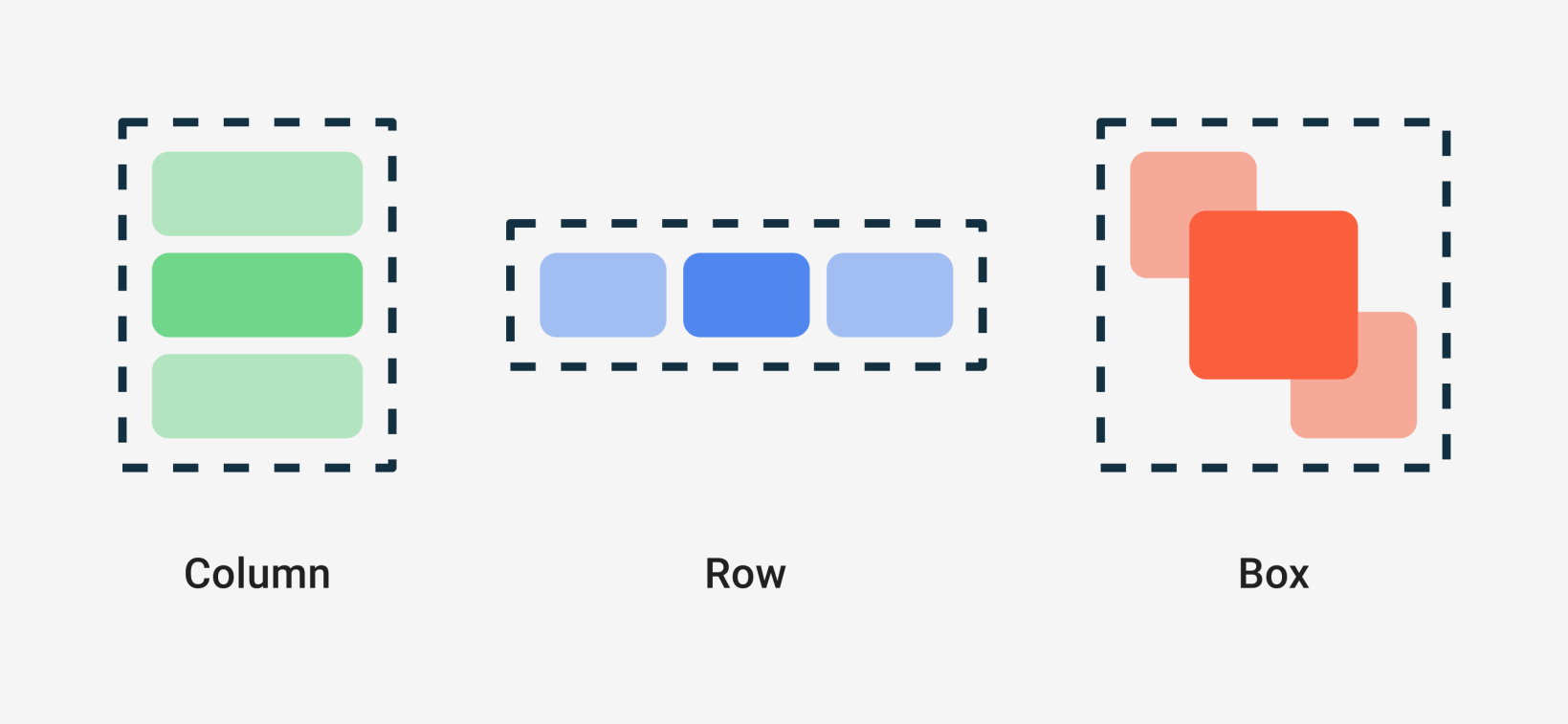
```
@Composable
fun HeroCard() {
    Column {
        Text("Рыцарь")
        Text("Уровень: 1")
    }
}
```

Column принимает лямбду содержимого.

Внутри неё вызываем другие функции UI.

Это Kotlin: вызовы функций, параметры и лямбда за скобками.

Column, Row и Box



Column — вертикально. Row — горизонтально. Box — слои и выравнивание.

Modifier: размер, отступы и поведение

```
Column(
    modifier = Modifier
        .fillMaxWidth()
        .padding(16.dp),
    verticalArrangement =
        Arrangement.spacedBy(12.dp)
) {
    Text("Рыцарь")
    Text("Здоровье: 100")
}
```

fillMaxWidth:
занять доступную
ширину.

padding: отступ.

spacedBy: расстояние
между элементами.

Modifier передаётся тому компоненту, который нужно настроить.

Порядок Modifier имеет значение

```
// A: зелёный фон включает отступ
Modifier
    .background(Color.Green)
    .padding(16.dp)

// B: отступ остаётся снаружи фона
Modifier
    .padding(16.dp)
    .background(Color.Green)
```

В A фон оборачивает область с отступом.

В B фон применяется к внутренней области.

Одинаковые операции в другом порядке могут дать другой результат.

dp, sp и доступное место

Запись	Для чего
16.dp	Размеры и расстояния с учётом плотности экрана
20.sp	Размер текста с учётом пользовательского масштаба
fillMaxWidth()	Доступная ширина от родителя
padding(16.dp)	Отступ вокруг содержимого
Arrangement.spacedBy(12.dp)	Промежуток между дочерними элементами

Размер экрана в пикселях не стоит зашивать в код интерфейса.

Material 3 и тема приложения

```
@Composable
fun MyApp() {
    MaterialTheme {
        Surface {
            HeroCard()
        }
    }
}
```

MaterialTheme задаёт цвета, типографику и формы.

Surface создаёт поверхность с цветом из темы.

В шаблоне обычно уже есть своя функция темы, например HeroTheme.

Scaffold и системные отступы

```
@Composable
fun MyApp() {
    MaterialTheme {
        Scaffold { innerPadding ->
            Column(
                Modifier.padding(innerPadding)
            ) {
                HeroCard()
            }
        }
    }
}
```

Scaffold — каркас экрана Material.

innerPadding помогает учесть его панели и системные области.

Полученные innerPadding нужно применить к содержимому.

Preview: компонент без ручного запуска

```
@Preview(showBackground = true)
@Composable
fun HeroCardPreview() {
    MaterialTheme {
        HeroCard()
    }
}
```

Android Studio показывает компонент в панели Preview.

Входные данные задаём прямо здесь.

Preview ускоряет работу с UI. Полный запуск на устройстве тоже нужен.

Кнопка и событие onClick

```
@Composable
fun HitButton() {
    Button(
        onClick = { println("Удар!") }
    ) {
        Text("Получить урон")
    }
}
```

onClick хранит действие, которое выполнится позже.

Text внутри Button описывает её надпись.

Лямбда onClick вызывается при нажатии, а не при описании кнопки.

Изменили переменную. Где новый текст?

var hp = 100

По нажатию выполнили hp -= 10.

Почему надпись «Здоровье: 100» может остаться прежней?

Обычная локальная переменная не решает задачу

```
@Composable
fun BrokenHealth() {
    var hp = 100

    Column {
        Text("Здоровье: $hp")
        Button(onClick = { hp -= 10 }) {
            Text("Получить урон")
        }
    }
}
```

Изменение hp
не наблюдается
Compose.

При новом выполнении
функции локальный hp
снова начнётся со 100.

Нужны наблюдаемое состояние и сохранение его объекта между выполнениями.

mutableStateOf и remember решают разные задачи

Средство	Его роль
mutableStateOf(100)	Создаёт наблюдаемое состояние со значением 100
state.value	Позволяет читать и менять значение состояния
remember { ... }	Сохраняет вычисленное значение в текущей композиции
remember { mutableStateOf(100) }	Сохраняет тот же объект наблюдаемого состояния

Наблюдать изменения и помнить значение — две отдельные задачи.

Рабочий экран со здоровьем

```
@Composable
fun HealthDemo() {
    val hp = remember { mutableStateOf(100) }

    Column {
        Text("Здоровье: ${hp.value}")
        Button(onClick = {
            hp.value = (hp.value - 10).coerceAtLeast(0)
        }) {
            Text("Получить урон")
        }
    }
}
```

hp хранит объект MutableState<Int>.

hp.value хранит текущее здоровье.

Меняем value в обработчике.

coerceAtLeast(0) не позволяет здоровью уйти ниже нуля.

Та же запись через by

```
var hp by remember { mutableStateOf(100) }

Text("Здоровье: $hp")

Button(onClick = {
    hp = (hp - 10).coerceAtLeast(0)
}) {
    Text("Получить урон")
}
```

by делегирует чтение и запись свойству value.

Вместо hp.value можно писать hp.

Механизм состояния тот же. Меняется удобство записи.

Что происходит после нажатия

Шаг	В нашем примере
1. Интерфейс читает состояние	Text использует hp = 100
2. Пользователь нажимает кнопку	Вызывается лямбда onClick
3. Обработчик меняет состояние	Новое hp = 90
4. Compose обновляет зависимый UI	Соответствующие функции могут выполниться снова
5. Пользователь видит результат	На экране «Здоровье: 90»

Рекомпозиция — повторное выполнение composable-функций для обновления UI.

Composable-функция может выполняться снова

- Описание интерфейса должно корректно работать при повторном вызове
- Урон наносим в onClick, а не прямо в теле функции экрана
- Запросы к сети и запись в файл не запускаем из тела composable
- Для управляемых побочных действий у Compose есть отдельные API

Повторное описание кнопки не должно само наносить новый урон.

TextField: ввод тоже меняет состояние

```
@Composable
fun NameInput() {
    var name by remember { mutableStateOf("") }

    TextField(
        value = name,
        onValueChange = { name = it },
        label = { Text("Имя героя") },
        singleLine = true
    )
}
```

value задаёт отображаемый текст.

onValueChange получает новое значение.

it — параметр лямбды.

Если обработчик не обновит name, новый ввод не закрепится в этом примере.

Кто должен хранить здоровье?

Надпись показывает hp.
Две кнопки меняют тот же hp.

Если каждый компонент заведёт своё состояние, что они будут показывать?

Подъём состояния

- Общий родитель хранит состояние, которое нужно нескольким компонентам
- Дочерний компонент получает текущее значение параметром
- О действиях пользователя он сообщает через функции-обработчики
- Один источник значения упрощает проверку и повторное использование

Состояние передаём вниз. События передаём вверх через callbacks.

Компонент получает данные и обработчик

```
@Composable
fun HealthPanel(
    hp: Int,
    onDamage: () -> Unit
) {
    Column {
        Text("Здоровье: $hp")
        Button(onClick = onDamage, enabled = hp > 0) {
            Text("Получить урон")
        }
    }
}
```

hp — готовое значение.

onDamage — функция без параметров и результата.

Своего hp здесь нет.

Компонент сообщает о нажатии. Родитель решает, как изменить здоровье.

Родитель связывает состояние и события

```
@Composable
fun HeroScreen() {
    var hp by remember { mutableStateOf(100) }

    HealthPanel(
        hp = hp,
        onDamage = {
            hp = (hp - 10).coerceAtLeast(0)
        }
    )
}
```

HeroScreen владеет здоровьем.

HealthPanel получает значение и действие.

На экране одно значение здоровья.

Компонент не обязан знать, где и как хранится значение.

Один компонент в разных состояниях

```
@Preview(showBackground = true)
@Composable
fun DefeatedHeroPreview() {
    MaterialTheme {
        HealthPanel(
            hp = 0,
            onDamage = {}
        )
    }
}
```

Можно посмотреть состояние hp = 0 сразу.

Для проверки вида не нужно нажимать кнопку десять раз.

Компонент с параметрами удобно показывать, проверять и использовать повторно.

Что сохраняет remember

Ситуация	Сохраняется ли значение?
Повторное выполнение в том же месте композиции	Да, remember возвращает сохранённое значение
Компонент покинул композицию	Его remembered-состояние забывается
Создан новый экземпляр Activity	Обычный remember не восстанавливает значение
Процесс завершён	Память процесса теряется

remember помогает при рекомпозиции. Восстановление состояния изучим дальше.

Частые ошибки в первом экране

Симптом	Что проверить
Две надписи наложились	Есть ли Column, Row или другой контейнер
Нажатие меняет число, но UI прежний	Используется ли наблюдаемое состояние
Здоровье неожиданно вернулось к 100	Где создаётся состояние и что его сохраняет
Урон наносится без нажатия	Не вызывается ли действие в теле composable
Не работает by	Есть ли импорты getValue и setValue

Сначала определяем: ошибка компиляции, поведения или размещения элементов.

Проверяем понимание

- Кто создаёт MainActivity и вызывает onCreate?
- Можно ли показать несколько экранов внутри одной Activity?
- Чем отличаются mutableStateOf и remember?
- Что произойдёт, если вызвать onDamage() при описании UI?
- Почему HealthPanel получает hp и callback отдельными параметрами?

Ответ должен объяснять поведение программы, а не только название API.

Теперь смотрим целое приложение

- Один персонаж и один экран
- Имя, текущее и максимальное здоровье
- Урон, лечение и начало заново
- От состояния до видимого результата



Сначала преподаватель показывает готовый проект, затем меняем его вместе.

Практика: карточка RPG-персонажа

- Добавить ввод имени и показывать его над характеристиками
- Кнопка урона уменьшает здоровье, лечение увеличивает
- Здоровье остаётся в диапазоне от 0 до 100
- При нуле показывать «Персонаж побеждён» и отключать урон
- Кнопка «Заново» восстанавливает здоровье до 100

Проверить ноль, максимум и несколько нажатий подряд.

Что дальше

- Пересоздание Activity и восстановление состояния
- rememberSaveable и ViewModel: разные задачи
- Состояние экрана и StateFlow
- Навигация между экранами внутри приложения

Сначала уверенно связываем данные, интерфейс и действия пользователя.

Материалы для повторения

- Android Developers: Introduction to activities
- Thinking in Compose и Compose layout basics
- State and Jetpack Compose, Where to hoist state
- Compose modifiers, Material 3, Composable previews

Примеры и ссылки на соответствующие темы есть в заметках к слайдам.